

Introduction au langage SQL

Vous retrouverez toutes les commandes utilisées dans ce TP, avec des exemples, à l'adresse suivante : <https://sql.sh/> (<https://sql.sh/cours/select>)

Définition :

SQL (*Structured Query Language*, « langage de requêtes structurées ») est un langage informatique normalisé servant à exploiter des bases de données relationnelles. La partie langage de manipulation des données de SQL permet de rechercher, d'ajouter, de modifier ou de supprimer des données dans les bases de données relationnelles.

Bonus : statistiques spécialités au lycée (<https://urlz.fr/qLv5>)

DOC 3 Le langage SQL

Le **SQL** (*Structured Query Language*) est un langage informatique permettant d'exploiter des données structurées en écrivant des requêtes. Les mots structurant les requêtes sont en anglais.

Mots réservés en SQL

Commandes SQL	Usage
select	Sélectionner une donnée
from	Dans une table
where	Avec une condition
order by	Trier le résultat dans l'ordre croissant
count	Compter
avg	Faire la moyenne

Commençons à nous intéresser à la table `personnage` .

A l'aide de la commande **SELECT** (<https://sql.sh/cours/select>), affichez le contenu de cette table.

```
Entrée[ ]: -- Tapez votre ligne de code ici
```

Vous remarquez que la table `personnage` est structurée ainsi :

idpersonnage	nom	surnom	nom_civil	idalignement	idunivers	sexe	puissance	idequiper
3	Black Widow	La veuve noire	Natasha Romanoff	1	1	F	40	1

`idpersonnage` `personnage` sert de *clé primaire* pour cette table.

Définition :

Dans une base de données relationnelle, une *clé primaire* est la donnée qui permet d'identifier de manière unique un enregistrement dans une table.

Attention !

La clé primaire doit toujours être unique ! Sinon cela entraîne une violation de l'intégrité des données. La clé primaire est essentielle pour identifier de manière unique chaque enregistrement dans une table, et elle doit donc être unique et non nulle.

Quelques requêtes sur une seule table

Requête simple

1. Que sait-on du personnage de nom `Hulk`? (pensez à utiliser *WHERE* (<https://sql.sh/cours/where/>))

Entrée[]:

Requête simple

2. Quels sont les noms des personnages féminins ? (19 résultats)

Entrée[]:

Requête simple

3. Quelles sont les femmes qui n'ont pas de nom civil connu ? (3 résultats)

Indice : Vous pourriez avoir besoin de AND (<https://sql.sh/cours/where/and-or>) et de IS NULL (<https://sql.sh/cours/where/is>)

Entrée[]:

Requête simple

4. Donner les noms des personnages féminins de l'équipe numéro 1. (3 résultats)

Entrée[]:

Requête simple

5. Afficher les noms des personnages des équipes 1 à 4 classés par numéro d'équipe et par sexe (28 résultats).

Indice : Vous pourriez avoir besoin de BETWEEN (<https://sql.sh/cours/where/between>) et de ORDER BY (<https://sql.sh/cours/order-by>)

Entrée[]:

Requête simple

6. Afficher les noms des personnages de l'équipe 1 classés par ordre alphabétique décroissant. (13 résultats).

Entrée[]:

Requête simple

7. Donner les noms des personnages dont le nom termine par *man*. (5 résultats)

Indice : Vous pourriez avoir besoin de LOWER() (<https://sql.sh/fonctions/lower>) et de LIKE (<https://sql.sh/cours/where/like>)

Entrée[]:

Requête simple

8. Donner les noms des personnages qui ne sont pas d'alignement 1 ni 3. (20 résultats)

Indice : Vous pourriez avoir besoin d'un peu de logique

Entrée[]:



Quelques requêtes utilisant des fonctions

COUNT() (<https://sql.sh/fonctions/agregation/count>)

1. Combien de personnages sont enregistrés ?

Entrée[]:

MAX() (<https://sql.sh/fonctions/agregation/max>)

2. Quelle est la plus grande puissance de personnage ?

Entrée[]:

MIN() (<https://sql.sh/fonctions/agregation/min>)

3. Quelle est la plus petite puissance de personnage ?

Entrée[]:

AVG() (<https://sql.sh/fonctions/agregation/avg>)

4. Quelle est la puissance moyenne des personnages ?

Entrée[]:

SUM() (<https://sql.sh/fonctions/agregation/sum>)

5. Quelle est la puissance cumulée de tous les personnages ?

Entrée[]:



Quelques requêtes sur plusieurs tables

Avant de pouvoir effectuer des requêtes sur plusieurs tables nous devons prendre connaissance de la structure globale de notre base de données.

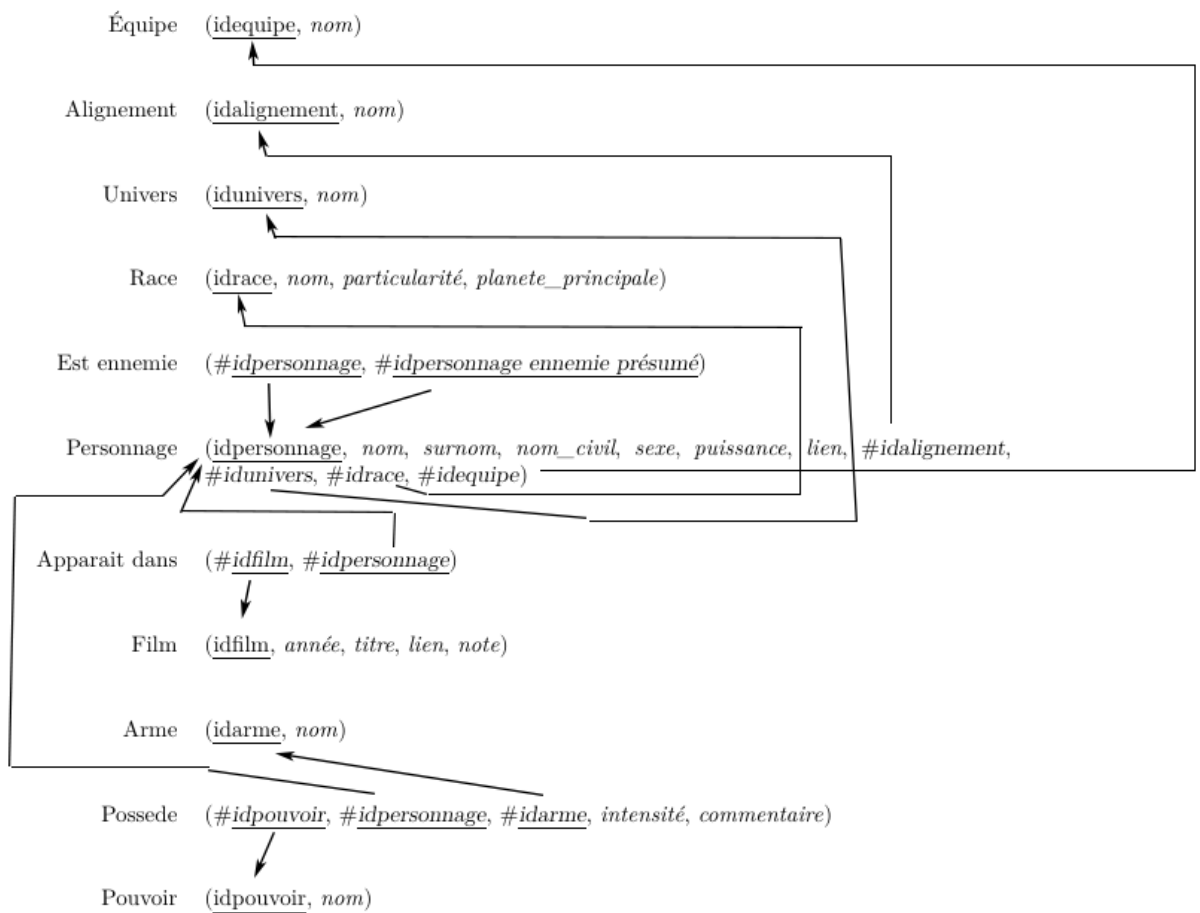
Équipe	(<u>idequipe</u> , <i>nom</i>)
Alignement	(<u>idalignement</u> , <i>nom</i>)
Univers	(<u>idunivers</u> , <i>nom</i>)
Race	(<u>idrace</u> , <i>nom</i> , <i>particularité</i> , <i>planete_principale</i>)
Est ennemie	(<u>#idpersonnage</u> , <u>#idpersonnage ennemie présumé</u>)
Personnage	(<u>idpersonnage</u> , <i>nom</i> , <i>surnom</i> , <i>nom_civil</i> , <i>sexe</i> , <i>puissance</i> , <i>lien</i> , <u>#idalignement</u> , <u>#idunivers</u> , <u>#idrace</u> , <u>#idequipe</u>)
Apparaît dans	(<u>#idfilm</u> , <u>#idpersonnage</u>)
Film	(<u>idfilm</u> , <i>année</i> , <i>titre</i> , <i>lien</i> , <i>note</i>)
Arme	(<u>idarme</u> , <i>nom</i>)
Possède	(<u>#idpouvoir</u> , <u>#idpersonnage</u> , <u>#idarme</u> , <i>intensité</i> , <i>commentaire</i>)
Pouvoir	(<u>idpouvoir</u> , <i>nom</i>)

Les champs soulignés sont des *clés primaires*, et les champs précédés d'un # sont des *clés étrangères*.

Définition :

La clé étrangère représente un champ (ou des champs) qui pointe vers la clé primaire d'une autre table.

Le modèle relationnel avec les liens :



Prenons un exemple :

Je souhaite connaître le nom de l'univers auquel appartient Black Widow. Pour obtenir cette information nous devrions commencer par trouver l' `idunivers` du personnage dans la table `personnage`

Example

```
SELECT idunivers FROM personnage
WHERE nom = 'Black Widow';
```

Nous trouvons : 1.

Ensuite, une seconde requête dans la table `univers` devrait nous permettre de répondre à la question :

Example

```
SELECT nom FROM univers
WHERE idunivers = '1';
```

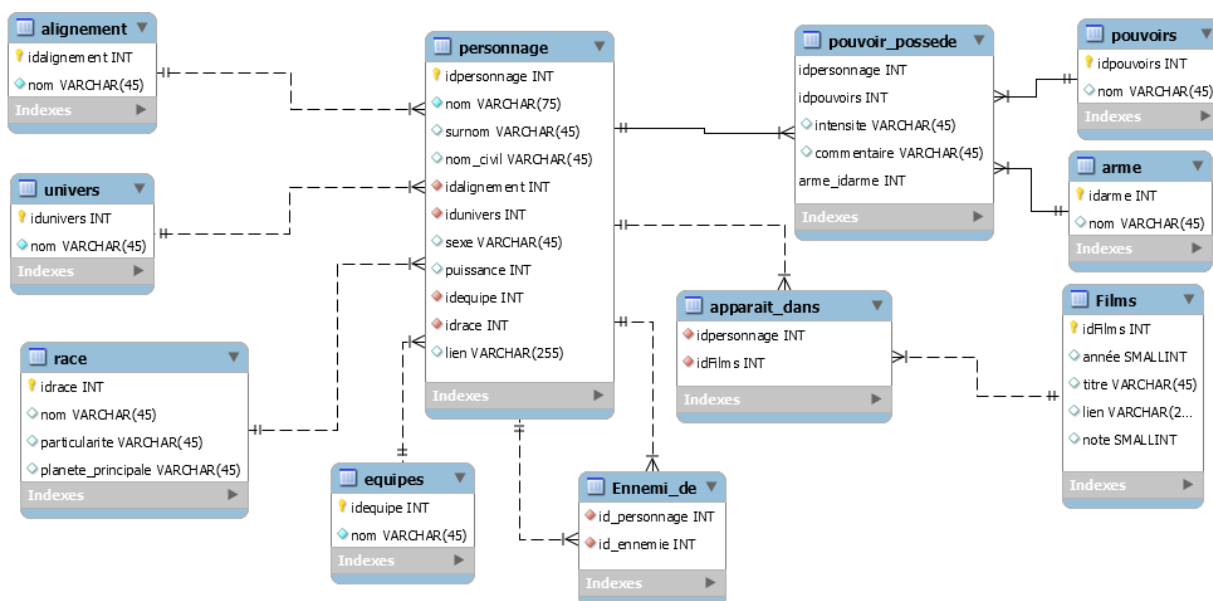
Et nous trouvons que Black Widow appartient à l'univers Marvel.

Il est possible de combiner ces deux requêtes en une seule, c'est ce qu'on appelle des requêtes imbriquées.

Exemple

```
SELECT nom FROM univers
WHERE idunivers = (SELECT idunivers FROM personnage
WHERE nom = 'Black Widow');
```

Enfin, une autre façon de modéliser la base de données est le modèle physique de données (MPD) :



A vous de jouer !!

Requêtes imbriquées

1. Quel est le nom de l'équipe d'Hulk ?

Entrée[]:

Requêtes imbriquées

2. Quelle est la particularité de la race du personnage `Ronan` ?

Entrée[]:

Requêtes imbriquées

3. Listes les noms des personnages qui ont un alignement mauvais par ordre alphabétique.

Entrée[]:

Requêtes imbriquées

4. Qui sont les membres de l'équipe d' `Hulk` ?

Entrée[]:

Requêtes imbriquées

5. Donner la liste des `Humains` , classés par `puissance` .

Entrée[]:

Requêtes imbriquées

6. Dans quels films apparait Thor ?

Astuce

Il peut être utile, parfois, d'utiliser : IN (<https://sql.sh/cours/where/in>)

En effet, le `=` suppose une valeur unique, parfois on cherche une valeur incluse dans un ensemble de valeurs.

Entrée[]:

Requêtes imbriquées

7. Donner la liste des personnages apparaissant dans le film Avengers ?

Entrée[]:

Requêtes imbriquées

8. Quels sont les pouvoirs de Hulk ?

Entrée[]:



Quelques requêtes sur plusieurs tables à l'aide de jointures

Jointure (<https://sql.sh/cours/jointures>)

Exemple

Donnons les personnages mauvais

```
SELECT personnage.nom FROM personnage
JOIN alignement ON alignement.idalignement = personnage.idalignement
WHERE alignement.nom = 'Mauvais';
```

Jointure

Quel est le nom de l'équipe d' Hulk ?

Entrée[]:

Jointure

Noms et race des personnages mauvais . *(16 lignes)*

Entrée[]:

Jointure

Donner la listes des personnages étant des Génie scientifique ou Génie intellectuel *(9 lignes)*.

Entrée[]:

Jointure

Listes des pouvoirs des personnages de l'équipe The Avengers *(17 lignes)*.

Entrée[]: